



디지털콘텐츠개론 2026 보충자료(nadopro@gmail.com)



정규식 (Regular Expression)

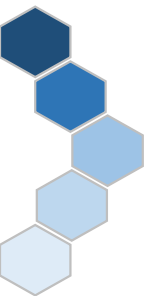
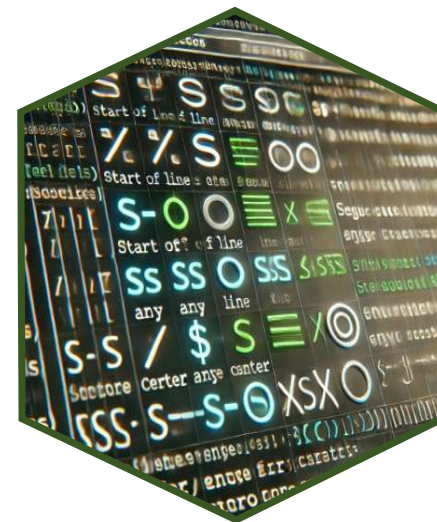
- 사용자 입력 값 검사에 사용하는 정규식
 - Pattern Matching Algorithm
 - 리눅스의 vi 편집기 명령, 콘솔 명령의 기초
 - PCRE (Perl Compatible Regular Expression)
 - 현존하는 패턴 매칭 알고리즘 중 성능이 가장 우수
 - 잘못된 곳의 위치를 찾는 것이 아니다.
 - 패턴에 맞는지 틀리는지 만 확인 (True / False)
 - 트래픽 모니터링, 백신 등에 사용





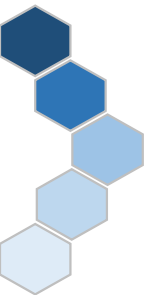
정규식 기초 문법 (기본 형태)

- $/^$: 정규식의 시작
- $/$: 정규식의 끝
- $\$$: 표현식의 끝
- $\backslash w$: 한 자리의 문자, 숫자 (word)
- $\backslash d$: 한 자리의 숫자 (decimal)
- $\backslash .$: 한자리의 문자



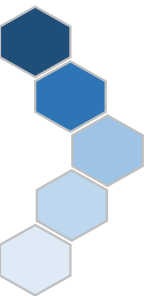


- * : ≥ 0
- + : ≥ 1
- ? : 0 or 1 time
- {n} : n times
- {min, max} : from min to max
- 예:
 - $\text{wd}\{3,4\}$: 숫자로 3 ~ 4회



정규식 기초 문법 (cont'd)

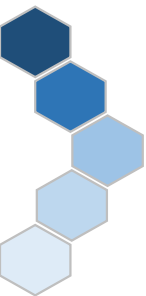
- g : Global의 약자, 패턴에 맞는 모든 문자 찾기
- i : ignore case, 대소문자 무시
- m : multiline, 여러 줄





정규식 기초 문법 (대괄호 [])

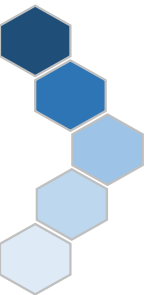
- [문자] : 대괄호 안의 문자 하나와 매칭되는가?
 - [abc] : a,b,c 중에 하나
 - [abc]{2} : a, b, c중에서 중복 허용해 2회
 - bc, ac, bb, cc, ca, aa, ba ...
 - 예: ls *.ch
- [^문자] : **not**, 괄호 안의 문자 제외
- [문자1-문자2] : 문자1과 문자2 사이의 값과 매치
 - 예: [a-z]는 a, b, c, d, e, ... , x, y, z와 매치
 - [abcdefghijklmnopqrstuvwxyz] 와 같은 표현





정규식 기초 문법 (특수 문자)

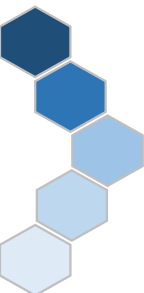
- \f : 폼피드
- \r : 캐리지 리턴
- \n : new line
- \t : tab
- \0 : null
- \b : back space
- \s : 공백 문자(space)
- \S : 공백이 아닌 문자
- \w : 알파벳 문자, 숫자
- \W : 알파벳 문자, 숫자가 아닌 문자
- \d : 정수(10진법, decimal)
- \D : 정수가 아닌 문자





정규식 기초 문법

- ID, Password 검사 예
 - `/^[a-z0-9]{4,10}$`
 - 영문소문자와 숫자로 구성된 4~10글자
- 휴대전화
 - `/^wd{3}-wd{3,4}-wd{4}$/`
 - 이렇게 검사하면 123-456-7890 도 휴대전화로 판단ㅎ
 - `/^01[016789]-[0-9]{4}-[0-9]{4}$/`
- 한글 이름 입력 검사
 - ???



정규식 활용

• JavaScript

제안된 예

```
<script>
function checkError() {
    var mobile = document.getElementById("mobile").value;
    var regex = /^(01[0|1|6|7|8|9])[-]?([0-9]{3,4})[-]?([0-9]{4})$/;
    if (!regex.test(mobile)) {
        alert("유효한 휴대전화번호를 입력해 주세요.");
        return false; // 폼 제출 방지
    }
    return true; // 폼 제출 허용
}
</script>
```



Question to ChatGPT

다음과 같은 사용자 입력값이 있을 때, 입력된 값이 휴대전화번호 패턴과 같은지 검사하는 자바스크립트를 제안해 줘.

```
<form onSubmit="return checkError()">
    <input type="text" name="mobile" id="mobile">
</form>
```



정규식 활용(Java)

제안된 예

```

1 import java.util.regex.Pattern;
2 import java.util.regex.Matcher;
3
4 public class PhoneNumberValidator {
5
6     public static boolean isValidPhoneNumber(String phoneNumber) {
7         // 정규식: 010, 011, 016, 017, 018, 019로 시작하며, 중간과 끝 부분이 각각 3~4자리 숫자인 패턴
8         String regex = "^([01[0|1|6|7|8|9])[-]?([0-9]{3,4})[-]?([0-9]{4})$";
9
10        // 정규식 패턴 컴파일
11        Pattern pattern = Pattern.compile(regex);
12        Matcher matcher = pattern.matcher(phoneNumber);
13
14        // 입력값이 정규식 패턴과 일치하는지 검사
15        return matcher.matches();
16    }
17
18    public static void main(String[] args) {
19        String testNumber1 = "010-1234-5678";
20        String testNumber2 = "01112345678";
21        String testNumber3 = "123-456-7890";
22
23        System.out.println(isValidPhoneNumber(testNumber1)); // true
24        System.out.println(isValidPhoneNumber(testNumber2)); // true
25        System.out.println(isValidPhoneNumber(testNumber3)); // false
26    }
27 }

```



Question to ChatGPT

다음과 같은 자바 코드를 제안해 줘.

- 사용자 입력값이 휴대전화 번호 형식이 맞는지 검사하는 정규식 검사.
- 패턴이 일치하면 TRUE, 일치하지 않으면 FALSE 출력





정규식 활용(C)



Question to ChatGPT

다음과 같은 c언어의 예를 제안해 줘.

- 정규식을 이용해 010-1234-5678 형식의 휴대폰 번호 패턴 검사
- argc !=2 일 때는 다음과 같은 형태의 메시지 출력후 종료
실행 파일 이름이 check라고 가정하면
[Usage] ./check 010-1234-5678
- argv[1]의 값을 검사해
입력전화번호 : Match 또는 Mismatch 출력
- 휴대전화 패턴은 #define MOBILE_PATTER에 정의
- 별도의 함수를 정의하지 않고, main()에서 바로 검사



제안된 예

```

1  #include <stdio.h>
2  #include <regex.h>
3  #include <stdlib.h>
4
5  #define MOBILE_PATTERN "^010-[0-9]{4}-[0-9]{4}$"
6
7  int main(int argc, char *argv[]) {
8      if (argc != 2) {
9          printf("[ Usage ] ./%s 010-1234-5678\n", argv[0]);
10         return 1;
11     }
12     regex_t regex;
13     int result;
14
15     // 정규식 컴파일
16     result = regcomp(&regex, MOBILE_PATTERN, REG_EXTENDED);
17     if (result) {
18         fprintf(stderr, "Could not compile regex\n");
19         return 1;
20     }
21

```

```

22     // argv[1]에 대해 정규식 검사 수행
23     result = regexec(&regex, argv[1], 0, NULL, 0);
24
25     // 결과 출력
26     if (!result) {
27         printf("입력전화번호 : Match\n");
28     } else if (result == REG_NOMATCH) {
29         printf("입력전화번호 : Mismatch\n");
30     } else {
31         char errorMsg[100];
32         regerror(result, &regex, errorMsg, sizeof(errorMsg));
33         fprintf(stderr, "Regex match failed: %s\n", errorMsg);
34         regfree(&regex);
35         return 1;
36     }
37
38     // 정규식 해제
39     regfree(&regex);
40     return 0;
41 }

```

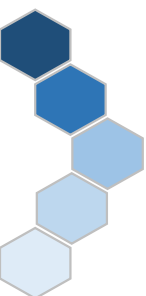
정규식 활용

- 나는 대한민국 사람이다.
 - 한글을 이용한 프로그램 개발
 - 환경설정
 - 데이터
 - 빅데이터에 사용된 한글 처리
 - 입력 값 검사
 - 예: 입력한 값이 한글 임을 보장하라





13



kpc 한국생산성본부

디지털콘텐츠개론 2026 보충자료(nadopro@gmail.com)

JSON 파일

- JSON 파일
 - JavaScript Object Notation
 - 인간이 읽기 쉽고, 기계가 쉽게 처리할 수 있는 데이터 형식
 - 구조화된 형식으로 표현하기 위한 표준화된 방식
 - 웹 어플리케이션에서 데이터 전송, 저장에 많이 사용



JSON의 주요 특징

- 경량 데이터 형식
 - 간결하고 직관적인 방식으로 표현
 - 텍스트 기반
 - 네트워크를 통해 데이터 전송에 유용
- 쉽고 읽기 쉬운 구조
 - 중괄호 { } : 데이터 객체
 - 대괄호 [] : 배열
- 언어 독립성

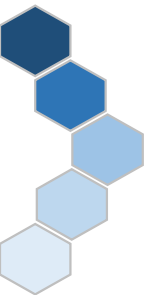


JSON 형식의 예

- 중괄호 : 객체
- 대괄호 : 배열
 - 이름 : 값 의 형식

JSON 파일의 예 : 중괄호 {}, 대괄호 []로 구분

```
{  
  "firstName": "John",  
  "lastName": "Doe",  
  "age": 30,  
  "isEmployed": true,  
  "skills": ["JavaScript", "Python", "SQL"],  
  "address": {  
    "street": "123 Main St",  
    "city": "Anytown",  
    "postalCode": "12345"  
  }  
}
```





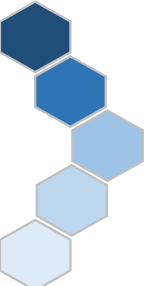
JSON 기본 문법

Case 1: 객체 하나를 표현

```
{
  "name": "홍길동",
  "age" : 22,
  "company" : "한국생산성본부"
}
```

Case 2: 객체 내부에 객체를 포함하는 경우

```
{
  "name": "홍길동",
  "age" : 22,
  "company" : {
    "name" : " 한국생산성본부",
    "url" : "https://kpc.or.kr"
  }
}
```





JSON 기본 문법

Case 3: 객체가 객체만 포함

```
{
  "person" : {
    "name": "홍길동",
    "age" : 22,
    "mobile": "010-1234-5678"
  },
  "company" : {
    "name" : " 한국생산성본부",
    "url" : "https://kpc.or.kr"
  }
}
```

Case 4: 배열 데이터

```
{
  "employee" : [
    { "name" : "홍길동", "age" : 22 },
    { "name" : "이순신", "age" : 45 },
    { "name" : "을지문덕", "age" : 56 }
  ],
  "employer": [
    { "company" : "한국대학교" },
    { "company" : "한국생산성 본부" }
  ]
}
```

JSON 기본 문법

JSON 활용 예

```
{
  "nodes" : [
    { "name": "홍길동" },
    { "name": "홍대감" },
    { "name": "정약용" },
    { "name": "추사" },
    { "name": "정약전" }
  ],
```

활용 예 (계속)

```
  "links" : [
    { "source": "홍길동", "target": "홍대감", "relation": "부자" },
    { "source": "홍길동", "target": "정약용", "relation": "친구" },
    { "source": "정약용", "target": "추사", "relation": "사제" },
    { "source": "정약용", "target": "정약전", "relation": "형제" }
  ]
}
```